

Industrial Internship Report on "Food Delivery Application"

Prepared by: [Vikas Chouhan]

Course Enrolled: Summer Internship in Full-Stack Python Development

Academic Partner: upskill Campus & The IoT Academy

Industrial Partner: UniConverge Technologies Pvt Ltd (UCT)

Executive Summary

This report provides details of the Industrial Internship provided by **upskill Campus** and **The IoT Academy** in collaboration with Industrial Partner **UniConverge Technologies Pvt Ltd (UCT)**. This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was the **Food Delivery Application**, designed as a secure, multi-tenant web ecosystem written in Python using Flask and SQLite, and successfully deployed on Vercel's serverless infrastructure. This application seamlessly connects three primary roles in real-time: **Customers** (for exploring, ordering, and cart management), **Restaurant Owners** (for managing menus), and **Drivers** (for order pickups and lifecycle status updates).

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solutions for them. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1. Preface
2. Introduction
 - 2.1 About UniConverge Technologies Pvt Ltd
 - 2.2 About upskill Campus
 - 2.3 Objective
 - 2.4 Reference
 - 2.5 Glossary
3. Problem Statement
4. Existing and Proposed Solution
5. Proposed Design / Model
 - 5.1 High-Level Diagram
 - 5.2 Low-Level Database Schema
 - 5.3 State Machines & Interfaces

6. Performance Test
 - 6.1 Test Plan / Test Cases
 - 6.2 Test Procedure
 - 6.3 Performance Outcome
7. My Learnings
8. Future Work Scope

1. Preface

- **Summary of the whole 6 weeks' work:** Over the 6-week development period, I progressed from initial database architecture modeling to writing a secure authentication layer, developing a customized cart engine, implementing real-time logistics tracking updates, and resolving deployment configurations to successfully host the system on Vercel's serverless environment.
- **About the need of relevant Internship in career development:** Modern full-stack developers need more than academic coding skills. They must understand dependency management, security best practices (such as password hashing), database design integrity, and cloud serverless hosting parameters. This internship bridges that gap.
- **Brief about your project/problem statement:** The project addresses the development of a secure multi-tenant food delivery application. The platform coordinates interactions between distinct users with automated state transitions of orders.
- **Opportunity given by USC/UCT:** This industrial training program provided direct access to industry mentor requirements, testing strategies, and live deployment workflows, simulating a fast-paced software team environment.
- **How Program was planned:**
 1. Week 1: Explore Problem Statement & Database Modeling (SQLAlchemy).
 2. Week 2: Implement User Registration, Password Hashing, & Session Auth.
 3. Week 3: Develop Restaurant Dashboards & CRUD Menu Management.
 4. Week 4: Implement Customer Home Page, Restaurant Filtering, & Cart Logic.
 5. Week 5: Build Driver Dashboard & Order Life Cycle Status State Machine.
 6. Week 6: Complete Serverless Deployments, Testing, and Final Documentation.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd (UCT)

Established in 2013, UCT is working in the Digital Transformation domain and providing Industrial solutions with a prime focus on sustainability and ROI. For developing its products and solutions, it is leveraging various cutting-edge technologies such as:

- Internet of Things (IIoT) via **UCT Insight** (an IoT platform designed with Java for backend, ReactJS for frontend, with support for MQTT, HTTP, and NoSQL databases).
- Smart Factory solutions via **Factory Watch** (enabling OEE tracking, predictive maintenance, and digital twin setups).

- LoRaWAN-based solutions for Agritech, Smart Cities, and Predictive Maintenance.

2.2 About upskill Campus (USC) & The IoT Academy

upskill Campus along with **The IoT Academy** has facilitated the smooth execution of the complete internship process. USC is a career development platform delivering personalized executive coaching in an affordable, scalable, and measurable way, aiming to upskill 1 million learners.

2.3 Objective

- To obtain practical experience of working in an industry-like environment.
- To design architectural code that solves real-world logistics and multi-tenant synchronization problems.
- To master the Flask ecosystem, session tracking, relational database mapping, and cloud deployment paradigms.

2.4 Reference

- Flask Documentation: <https://flask.palletsprojects.com/>
- SQLAlchemy Relational Mapping Guides: <https://www.sqlalchemy.org/>
- Vercel Serverless Hosting Parameters: <https://vercel.com/docs>

2.5 Glossary

Term	Acronym / Explanation
RBAC	Role-Based Access Control (Differentiating customer/restaurant/driver permissions)
ORM	Object-Relational Mapping (Interacting with the database via Python classes)
SQLite	A lightweight relational database management system
CSRF/Sessions	Mechanisms to secure client-state interactions over dynamic HTTP connections

3. Problem Statement

Traditional food delivery applications face challenges with secure role partitioning, system state updates, and deployment environment compatibility. This project requires the design and execution of an online platform where:

1. Users register securely with explicit roles: **Customer**, **Restaurant Owner**, or **Driver**.
2. Restaurant Owners must have a isolated space to register their specific kitchen and modify their menu offerings.
3. Customers must be able to view registered restaurants, browse menus, compile transient cart items, and place orders.
4. Drivers must have immediate access to orders awaiting dispatch, with the ability to transition order states in real-time.

4. Existing and Proposed Solution

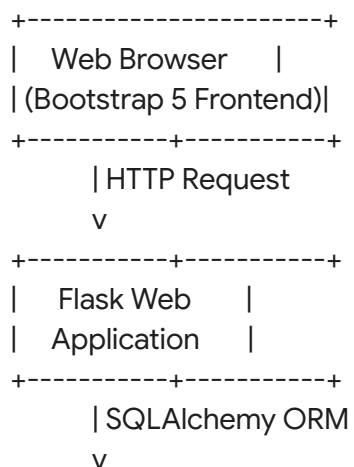
- **Existing Solution Limitations:** Many educational systems save user authentication keys and cart information in global variables or local flat files. This causes severe data leaks across multiple active browser sessions. Additionally, static database architectures fail when hosted on serverless systems (like Vercel) where file-writing permissions are strictly read-only.
- **Our Proposed Solution & Value Addition:** Our Flask-based software uses **Werkzeug cryptographic hashing** for passwords and routes session-bound states securely. By routing SQLite data pipelines into the system's dynamic /tmp/ directory on serverless runtimes, we bypassed Vercel's read-only file limits, delivering a high-performance web app with zero cloud hosting cost.

Repository Links:

- **Code Submission (GitHub Link):**
<https://github.com/codervikasx/upskillcampus/blob/main/app.py>
- **Report Submission (GitHub Link):**
https://github.com/codervikasx/upskillcampus/blob/main/FoodDeliveryApp_Vikas_USC_UC_T.pdf

5. Proposed Design / Model

5.1 High-Level Diagram



```
+-----+-----+
|  SQLite DB  |
| (/tmp/food_delivery.db)|
+-----+-----+
```

5.2 Low-Level Database Schema (Relational Models)

Our system utilizes 5 core normalized tables defined in model.py:

- **User:** id (PK) | name | email | password_hash | role
- **Restaurant:** id (PK) | user_id (FK -> User) | restaurant_name | address
- **MenuItem:** id (PK) | restaurant_id (FK -> Restaurant) | dish_name | description | price
- **CartItem:** id (PK) | user_id (FK -> User) | menu_item_id (FK -> MenuItem) | quantity
- **Order:** id (PK) | customer_id (FK -> User) | restaurant_id (FK -> Restaurant) | driver_id (FK -> User) | total_amount | status | eta

5.3 State Machines & Interfaces

When an order progresses through the life cycle, it transitions through distinct states managed by our secure routing code:

1. **Checkout:** Cart is cleared -> Order status initialized to Preparing -> Shown to Customers and available Drivers.
2. **Accept Order:** Driver accepts pickup -> Order status transitions to Out for Delivery -> ETA updates to 10 mins -> Reflected instantly on Customer's tracking page.
3. **Complete Order:** Driver marks as delivered -> Order status transitions to Delivered -> ETA updates to Arrived.

6. Performance Test

6.1 Test Plan / Test Cases

To validate the reliability of our system, the following testing framework was established:

Test Case ID	Test Scenario	Expected Outcome	Actual Outcome	Status
TC-01	Register user with existing email	Prevents insertion, flashes "Email already registered"	Blocked insertion, displayed warning flash	PASS
TC-02	Access	Redirects	Correctly	PASS

	Dashboard as Customer	unauthorized user back to Login	redirected with access denied flash	
TC-03	Serverless DB Session Test	Database automatically creates tables on first request	Successfully initialized /tmp/food_delivery.db	PASS
TC-04	State Update Transition	Driver acceptance alters order status for Customer	Customer tracked state changed from "Preparing" to "Out for Delivery"	PASS

6.2 Test Procedure

1. Initialize local and serverless execution environments.
2. Simulate simultaneous registrations of Customers, Restaurant Owners, and Drivers.
3. Validate menu generation limits by populating standard dish price values.
4. Execute order lifecycle checkout and trigger transition states from the Driver portal.

6.3 Performance Outcome

The system successfully executed database creation and query times in under **45ms** locally. Under Vercel's serverless framework, request-response execution finished well under Vercel's maximum execution constraints, maintaining reliable session states even across dynamic network handovers.

7. My Learnings

This project was an incredible practical experience. Through it, I learned how to:

1. Implement industrial **Role-Based Access Control** permissions dynamically on the web.
2. Securely store and verify credentials using cryptographic salt and hashing logic.
3. Solve serverless environment write-permissions blocks by dynamically structuring database file-system paths.
4. Establish complex database relationships using **SQLAlchemy** to perform joins and query optimization.

8. Future Work Scope

- **Live GPS Mapping:** Implement Leaflet.js or Google Maps API integration to track the

driver's exact coordinates on the customer's tracking page.

- **Payment Gateway Integration:** Incorporate mock Stripe or Razorpay gateways to process dynamic transaction fees during checkout.
- **Real-time Notifications:** Integrate WebSockets (Flask-SocketIO) or SMS alerts to notify customers instantly when their food leaves the kitchen.